

改进后的软件过程定义文档

一、过程概述

1.1 项目背景

本项目为校园闲置物品交易小程序，主要面向高校学生用户，提供闲置商品发布、浏览、在线沟通、订单管理以及校园自提预约等功能。

项目技术架构如下：

- 前端：微信小程序
- 后端：Spring Boot
- 数据库：MySQL
- 部署环境：云服务器

原软件过程采用“瀑布模型 + 敏捷开发思想”的轻量化开发模式，共包含需求分析、系统设计、开发实现、测试验证、部署上线以及运维迭代六个阶段。

随着生成式 AI、大语言模型（LLM）以及 AI Agent 技术的发展，软件开发逐渐由传统人工驱动向人机协同开发转变。

因此，本项目在原有软件过程基础上引入 AI 能力，构建 AI 增强型软件过程体系。

1.2 过程目标

改进后的软件过程主要目标如下：

提高开发效率

利用 AI 自动生成文档、代码和测试用例，减少重复劳动。

目标：

- 整体效率提升 40%~60%

提高软件质量

利用 AI 进行需求检查、代码审查和测试覆盖分析。

目标：

- 缺陷率下降 30%以上
-

提升过程规范性

通过标准化 Prompt 模板和自动化文档生成，提高过程一致性。

降低单人开发成本

借助 AI 承担部分产品经理、架构师和测试工程师职责。

二、过程总体架构

2.1 生命周期模型

改进后的软件生命周期如下：

flowchart TD

A[需求调研]

-->B[AI 需求分析]

B

-->C[需求评审]

C

-->D[AI 架构设计]

D

-->E[原型设计]

E

-->F[AI 辅助开发]

F

-->G[AI 代码审查]

G

-->H[AI 测试生成]

H

-->I[自动化测试]

I

-->J[部署上线]

J

-->K[AI 运维监控]

K

-->L[版本迭代]

2.2 过程特点

与原过程相比：

新增：

- AI 需求分析
- AI 架构设计
- AI 代码生成
- AI 测试生成
- AI 运维分析

形成：

“开发者负责决策，AI 负责辅助执行”的协同模式。

三、角色定义

3.1 开发者

职责

负责：

- 需求确认
- 架构决策
- 核心业务实现
- AI 结果审核
- 项目交付

能力要求

- Java 开发能力
- 数据库设计能力
- 软件工程知识

3.2 AI 过程工程师

职责

负责：

- AI 工具管理
- Prompt 模板设计
- AI 结果质量监督
- AI 知识库维护

输出物

- Prompt 模板库
- AI 输出评估报告

3.3 AI 测试分析师

职责

负责：

- AI 测试用例审核
- 覆盖率分析
- 测试策略优化

输出物

- 测试覆盖率报告
 - 测试质量分析报告
-

四、需求分析阶段

4.1 阶段目标

明确用户需求。

形成项目开发范围。

4.2 输入

- 用户调研结果
 - 项目目标描述
-

4.3 核心活动

用户需求调研

收集学生用户痛点：

- 商品交易
 - 商品搜索
 - 在线沟通
-

AI 需求生成

使用 ChatGPT 生成：

- 用户故事
 - 功能需求
 - 非功能需求
-

AI 需求检查

检查：

- 需求冲突
 - 重复需求
 - 缺失需求
-

需求评审

人工审核最终需求。

4.4 输出物

输出物

需求说明书

用户故事清单

需求一致性报告

项目计划表

4.5 完成标准

- 核心功能明确
 - 无冲突需求
 - 需求覆盖完整
-

五、系统设计阶段

5.1 阶段目标

完成系统架构设计与数据库设计。

5.2 输入

- 需求说明书
 - 用户故事
-

5.3 核心活动

AI 架构设计

生成：

- 系统架构图
 - 模块划分图
-

原型设计

设计：

- 首页
 - 商品页
 - 聊天页
 - 订单页
-

数据库设计

设计：

- 用户表
 - 商品表
 - 订单表
 - 聊天表
-

UML 建模

生成：

- 用例图
 - 类图
 - 时序图
-

5.4 输出物

输出物

架构设计文档

原型图

数据库设计文档

UML 模型

5.5 完成标准

- 架构设计合理
 - 数据结构完整
 - 原型覆盖全部流程
-

六、开发实现阶段

6.1 阶段目标

完成系统功能开发。

6.2 输入

- 设计文档
 - 数据库设计
-

6.3 核心活动

AI 代码生成

利用：

- Cursor
- GitHub Copilot

生成：

- Controller
 - Service
 - Mapper
-

模块开发

按照顺序：

1. 用户模块
 2. 商品模块
 3. 聊天模块
 4. 订单模块
-

AI 代码重构

优化：

- 冗余逻辑
 - 重复代码
 - 性能问题
-

Git 版本管理

按功能提交代码。

6.4 输出物

输出物

源代码

Git 提交记录

AI 代码生成记录

6.5 完成标准

- 模块开发完成
 - 编译通过
 - 无严重缺陷
-

七、测试验证阶段

7.1 阶段目标

验证系统功能正确性。

7.2 输入

- 系统代码
 - 接口文档
-

7.3 核心活动

AI 生成测试用例

覆盖：

- 正常流程
 - 异常流程
 - 边界场景
 - 安全场景
-

自动生成测试脚本

生成：

- JUnit 测试
 - 接口测试脚本
-

集成测试

验证：

发布商品

↓

在线沟通

↓

下单

↓

完成交易

用户灰度测试

邀请同学试用。

收集反馈。

7.4 输出物

输出物

测试用例库

自动化测试脚本

测试报告

用户反馈记录

7.5 完成标准

- 核心流程测试通过
 - 覆盖率达到 90%以上
-

八、部署上线阶段

8.1 阶段目标

完成系统部署与上线。

8.2 输入

- 测试通过版本
-

8.3 核心活动

服务器部署

部署 Spring Boot 服务。

数据库部署

部署 MySQL。

HTTPS 配置

配置 SSL 证书。

微信审核

提交微信公众平台审核。

AI 运维预检查

检查：

- 配置错误

- 安全漏洞
 - 性能瓶颈
-

8.4 输出物

输出物

上线版本

部署文档

用户手册

8.5 完成标准

- 系统可正常访问
 - 微信审核通过
-

九、运维与迭代阶段

9.1 阶段目标

保障系统稳定运行。

9.2 输入

- 线上日志
 - 用户反馈
-

9.3 核心活动

AI 日志分析

自动发现：

- 500 错误
- 慢查询

- 内存异常
-

根因分析

识别:

- 数据库问题
 - 网络问题
 - 程序问题
-

Bug 修复

及时处理问题。

功能优化

持续迭代:

- 商品推荐
 - 信用评价
 - 智能搜索
-

9.4 输出物

输出物

运维日志

异常分析报告

迭代计划

9.5 完成标准

- 系统稳定运行
- Bug 响应时间小于 24 小时

十、质量保证机制

10.1 AI 输出审核机制

所有 AI 输出必须经过人工审核。

包括：

- 需求文档
- 设计文档
- 代码
- 测试用例

10.2 双重验证机制

AI 生成

↓

人工审核

↓

测试验证

↓

正式采用

10.3 缺陷管理机制

发现缺陷后：

1. 记录问题
 2. 分析原因
 3. 修复问题
 4. 回归测试
-

十一、过程度量指标

11.1 开发效率指标

指标	目标
需求分析效率提升	≥ 70%
设计效率提升	≥ 50%
编码效率提升	≥ 40%
测试效率提升	≥ 80%

11.2 软件质量指标

指标	目标
测试覆盖率	≥ 90%
严重缺陷数	≤ 3
线上故障恢复时间	≤ 1 小时

11.3 AI 贡献度指标

定义：

$$\text{AI 贡献度} = \frac{\text{AI 完成工作量}}{\text{总工作量}} \times 100\%$$

目标：

AI 贡献度 ≥ 40%

十二、风险控制

风险一

AI 生成错误内容。

措施：

人工审核。

风险二

敏感数据泄露。

措施：

数据脱敏。

风险三

过度依赖 AI。

措施：

保持人工决策主导。

风险四

AI 结果不稳定。

措施：

建立标准 Prompt 模板库。

十三、总结

本过程在原有轻量化软件开发流程基础上，引入生成式 AI、大语言模型以及智能测试技术，实现需求分析、系统设计、编码开发、测试验证和运维监控全生命周期智能化升级。

改进后的软件过程既保留了传统软件工程的规范性，又充分利用 AI 技术提升开发效率和软件质量，形成了适用于个人项目与中小型项目开发的 AI 增强型软件过程体系。

预计整体开发效率提升 40%~60%，测试覆盖率提升至 90%以上，为未来智能软件工程实践提供有效参考。